

数据结构 CS1652

郑冠杰

2026 年 3 月 2 日

① 什么是数据结构

- ① 数据结构定义
- ② 数据结构研究内容

② 算法及算法分析

③ C++ 部分概念

什么是数据结构

- **数据：**

- 外界信息进入计算机后，统称为数据。一般分为数值和非数值型。

- **数据元素：**

- 是数据处理的最小单位，是一个数据个体。如处理某窗口前的队列，队列中所有人的信息称数据，每个人的信息称数据元素。

- **数据结构：**

- 有限个、类型相同、相互之间具有一定制约关系的数据元素组成的集合。如某窗口前的队列，有限/类型相同/你先我后关系。

数据结构的研究内容

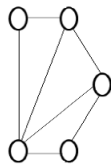
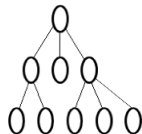
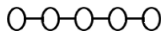
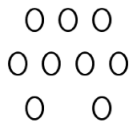
- 数据结构的研究内容：

- （逻辑结构）+ 基本操作（关系操作）——来源于生活实践，和计算机无关。
- 存储实现（物理结构）——冯·诺依曼体系结构。数据及数据关系在内存中的存储，要有利于基本操作实现。
- 基本操作的实现——某种存储处理下各种基本操作的实现。
- 典型应用——这种数据结构在生活实践中的典型应用。

数据的逻辑关系（逻辑结构）

指类型相同的一组元素和元素间的关系。根据关系不同，可分为以下几种：

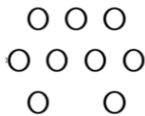
- 集合关系：不同元素除了同属于一个集合，相互间无其他制约关系。
- 线性关系：元素间呈现你先我后的顺序，是一种一对一的关系。
- 树形关系：元素间呈现一对多的关系。
- 图关系：元素间呈现多对多的关系。



数据的逻辑关系（逻辑结构）

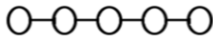
- 集合结构

元素间的次序是任意的。不同元素除了同属于一个集合，相互间无其他制约关系。



- 线性结构

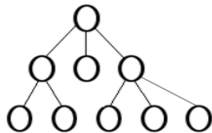
数据元素的有序序列。除了第一个和最后一个元素外，其余元素都有一个直接前趋和一个直接后继。元素间呈现你先我后的顺序，是一种一对一的关系。



数据的逻辑关系（逻辑结构）

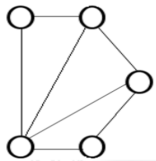
- 树形结构

除根元素外，每个节点有且仅有一个前趋，后继数目不限。元素间呈现一对多的关系。



- 图型结构

每个元素的前趋和后继数目都不限。元素间呈现多对多的关系。



逻辑结构的描述

逻辑结构通常可以用二元组描述: $\text{Data_Struct}=(D,R)$, 其中 D 是元素集合, R 是关系集合。

例如整数 1~10 组成的有序集就是一个线性结构, 它可表示为:

$D = \{x | 1 \leq x \leq 10, x \in \mathbb{N}\}$, $R = \{ \langle x_1, x_2 \rangle \mid x_1 \in D, x_2 \in D \}$ 。其中 $\langle x_1, x_2 \rangle$ 表示一个有序偶, 即 x_1 和 x_2 有顺序关系, x_1 是前驱, x_2 是后继。

逻辑结构是事物本身的性质, 和计算机没有关系。

基本操作 (关系操作)

具有某种关系的数据在生活实践中表现出的几种功能相对独立的数据处理（操作）。

- 它和数据的逻辑结构紧密相关，它来源于现实生活中关系自身的特点。
- 无论哪种逻辑结构，基本操作都可分为五大类：**构造类、属性类、数据操纵类、遍历类和典型应用类。**

基本操作 (关系操作)

- **构造类**：在内存中建立这种数据结构。如一个队列，有存储空间，无或有若干元素。
- **属性类**：对元素及元素之间关系的各类查询。属于东瞧瞧、西看看，不影响元素值及元素关系。如在线性结构中查询值为 X 的元素是否存在，队列中队首是谁。
- **数据操纵类**：对元素或元素关系有改变的操作。如插入或删除某个元素，一般修改可以视作在同一位置上先删除一个旧元素后再插入一个新元素，因此不再讨论修改。

基本操作 (关系操作)

- **遍历类**：对结构中的每个元素访问且只访问一遍。因其重要且有时又较复杂，常常是其他操作的基础，如遍历树结构、图结构中的元素，所以特意把遍历操作从属性类中单独拿出来研究。
- **典型应用类**：每种结构独特的应用。不同结构其典型应用各不相同，如线性结构可以解决队列问题、图结构可以解决两个城市间最短路径问题。

数据结构的基本操作

- **创建**：创建一个数据结构
- **清除**：删除数据结构
- **插入**：在数据结构指定的位置上插入一个新元素
- **删除**：将数据结构中的某个元素删去
- **搜索**：在数据结构中搜索满足特定条件的元素
- **更新**：修改数据结构中的某个元素的值
- **访问**：访问数据结构中的某个元素
- **遍历**：访问数据结构中的每一元素，使每个元素恰好被访问一次
- 每一种数据结构的特定操作

什么是存储结构

数据结构在内存中的表示。

- 数据要得到处理首先必须进入内存。内存中不仅要存储数据元素的值，还要存储元素间的关系。
- 任何一种数据结构元素关系和基本操作的设计取决于其逻辑结构，但基本操作的实现完全依赖其存储结构。
- 元素及其关系在内存中用什么结构存储适合，原则是存储方式要有利于基本操作的实现。

数据结构的存储实现（物理结构）

- 包括两个部分：
 - 数据元素的存储
 - 数据元素之间的关系的存储
- 存储方式：
 - 顺序存储：用存储的位置表示元素之间的关系。高级语言中，主要用数组实现。
 - 链接存储：用指针显式地指出元素之间的关系，如单链表就是表示线性关系的。

常见的存储方式

顺序存储：

用一块连续的空间存储数据，借助空间地址上的有序性存储元素间的关系。顺序存储的结构以下称**顺序结构**。

高级语言中，数组是地址连续的空间，有助于实现顺序结构。

常见的存储方式

链式存储：

元素可以各自存储在独立的空间中，不要求不同数据存储的内存空间连续，元素间的关系附带存储在数据各自占据的独立空间中。

数据结构的链式存储也称链式结构。

基本操作的实现

- 存储数据及其关系的目的是为了处理。
- 基本操作实现方法以存储方法为基础。如果是顺序结构，操作实现就是对数组做各种不同的操作；如果是链式结构，操作实现就是对链表做各种不同的操作。
- 同一种操作虽然在顺序结构和链式结构中具体实现方法不同，但目标都要符合 ADT 对基本操作定义的条件和结果。
- 如果某种存储方式下，基本操作不易实现，说明存储方式不好，可以考虑放弃这种存储方法。

① 什么是数据结构

② 算法及算法分析

- ① 算法及算法特性
- ② 算法时间复杂度
- ③ 算法空间复杂度
- ④ 算法的优化

③ C++ 部分概念

算法：解决问题的方法和步骤

- 数据结构最终目标就是讨论一组数据的处理问题。
- 算法和存储方式紧密相连。
- 每种操作有多种实现方法（算法）。
- 算法有优有劣。

算法的特性

- **确定性**：每一步都有确定的含义，没有二义性。
- **有穷性**：每一步运行有限时间，有限步可完成。
- **可行性**：每一步经过有限次基本操作可完成，每一步自身没有复杂算法。
- **有输入**：有零个或若干输入作已知条件。
- **有输出**：有零个或若干输出作算法结果。

算法的基本要求

- **正确性**：准确反映并能满足具体问题的要求；
- **可读性**：可供人们阅读的程度；
- **健壮性**：对各种不同的输入都要有相应的反应。
- **时间效率**：算法的执行时间；
- **空间效率**：算法执行期间所需要的最大内存空间。

算法的质量评价

如何确定一个算法是高效的算法就是分析该算法所需要的资源。算法的资源包括：

- **时间**：程序运行所需要的时间。要运行一年的算法是很难让人接受的。OJ 上时限与复杂度的对应大概是 $1000ms$ 对应 10^9 的复杂度
- **空间**：程序运行所需要的空间。需要几个 G 的内存的算法同样也令人难以接受。一个 *int* 型变量占 $32bit = 4byte$ ，一个 *double* 型变量占 $64bit = 8byte$

设计算法的误区

设计算法的误区：从以往生活的经验出发，找解决问题的方法

- **比如：**选最大桔子问题，体育课排队，5 个数字的加法。
- **正确的方式：**计算思维，简单 + 反复，具体从编程语言支持的操作出发。
- **具体编程语言中的操作：**输入、输出、赋值、两两比较、算术和逻辑运算，循环、函数（递归）等。

如冒泡排序，选择排序的设计思想
生活中基本不用计算思维。

运行前分析的依据

- ① 机器的运算速度：一般根据主频和字长。
- ② 编译后代码的质量：因编译优化策略不同，运行效率不同。
- ③ 书写程序所用的语言：语言越高级，运行效率低，编程效率高。
- ④ 问题的规模和数据分布：规模大，分布特殊则处理方法可不同。
- ⑤ 算法采用的策略和方法：具体采用的方法，如用迭代或递归就不同。

算法设计主要关注 5，**算法的策略和方法**。

算法运算量的计算

- 选择几种操作作为“标准操作”。
- 将标准操作作为一个抽象的运算单位。
- 计算算法共执行了多少次标准操作，次数即计算量。
- 计算量是一个元素规模的函数，称**时间频度**。

如果有一组正整数，存放在数组 *array* 中，要求设计一个算法求数组中的最大值与 d 的乘积。

实例

Max1

```
int max1(int *array, int n, int d)
{int max=0, i;
  for (i=0; i < n ; ++i)
    array[i] *= d;
  for (i=0; i < n ; ++i)
    if (array[i] > max)
      max = array[i];
  return max;
}
```

实例

max1:

- 第一个 for 循环:

第三行中, 表达式 1 执行一次, 表达式 2 执行 $n+1$ 次, 表达式 3 执行了 n 次。第四行是循环周期, 每个循环周期执行一次乘法、一次赋值。总的运算量为 $1 + (n+1) + n + n*2 = 4n + 2$ 。

- 第二个循环:

第五行中, 表达式 1 执行了一次, 表达式 2 执行了 $n+1$ 次, 表达式 3 执行了 n 次, 第六行和第七行是循环周期, 每个循环周期执行一次比较, 一次赋值。总运算量为 $1 + (n+1) + n + n*2 = 4n + 2$ 。

- 加上首尾两条语句, max1 总运算量 (时间频度) 是 $8n + 6$ 。

- 算法运行的时间函数可能是一个很复杂的函数，如何比较这些函数并从中选取出一个好的算法呢？
- 时间性能主要考虑的是：
问题规模很大的时候运行时间随问题规模的变化规律

渐进表示法

如果当 n 趋于无穷时, $T(n)/f(n)$ 的极限为一个非零常数, 称 $O(f(n))$ 为算法的渐进时间复杂度, 简称时间复杂度。这种形式称为大 O 表示法。

- 渐进表示法:

不考虑具体的运行时间函数, 只考虑运行时间函数的数量级

- 定义: 大 O 表示法 (O 是 order, 数量级)

如果存在正的常数 C 和 N_0 , 满足当 $N \geq N_0$ 时有: $T(N) \leq Cf(N)$, 则 $T(N)$ 是 $O(f(N))$ 。

- 用公式表达为 $\exists C$ 和 $N_0 \forall n > N_0$, 有 $T(n) \leq C * f(n)$, 则 $O(f(n))$ 为算法的时间复杂度。

算法的时间复杂度

例子 1:

$$T(n) = 3n^2 + 2n + 10$$

$$T(n) \leq 3n^2 + 2n^2 + 10n^2 \leq 15n^2$$

存在 $N_0 = 1, C = 15$, 当 $n > N_0$ 时, 有 $T(n) \leq 15 * n^2$, 即 $f(n) = n^2$, 算法的复杂度为 $O(n^2)$ 。

例子 2: 当 $T(n) = 3n + 3$ 时, $T(n) \leq 6n$, 时间复杂度为 $O(n)$

时间复杂度由时间频度函数中的最高次项决定, 不带系数。

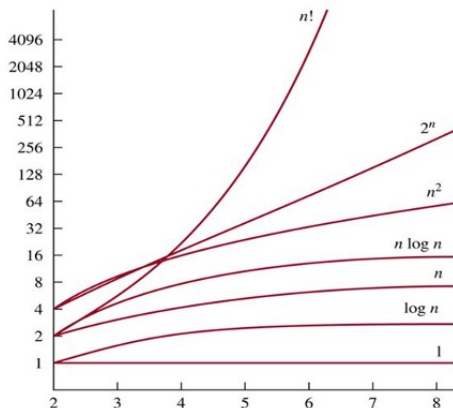
有效算法的重要性

计算机不是万能的。并非所有算法都能够计算出有用的结果。差的算法不一定有实际意义。如果计算机 1 秒能处理 1000 条指令，那么处理 n 个数据所需执行的指令数如表中的函数所示：

时间函数	$n = 20$	$n = 50$	$n = 100$	$n = 500$
n	0.02s	0.05s	0.1s	0.5s
$n \log n$	0.09s	0.3s	0.6s	4.5s
n^2	0.4s	2.5s	10s	250s
n^3	8s	2 分	17 分	35 小时
2^n	34 分	若干年	若干世纪	

典型的增长率或称复杂度

1	常量
$\log N$	对数
$\log^2 N$	对数的平方
N	线性
$N \log N$	$N \log N$
N^2	平方
N^3	立方
2^N	指数



复杂度不限于这几种，如 $n^{\frac{1}{2}}$

算法按时间复杂度分类

- 多项式时间：渐进时间复杂度有多项式时间限界的算法
- 指数时间算法：渐进时间复杂度有指数时间限界的算法
- 多项式时间复杂度的关系：

$$O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(N^3)$$

- 指数时间复杂度的关系：

$$O(2^N) < O(N!) < O(N^N)$$

算法的时间复杂度

- 通常考虑三个方面：最好、最坏和平均情况的时间复杂度。一般用最坏情况作为算法的时间复杂度。
- 大 O 表示法中， O 是数量级 order 的首字母。
- $O(f(n))$ 并不描述运行时间的精确值，只给出一个数量级。
它表示：当问题规模很大时，算法运行时间的增长受限于哪一个数量级的函数，简称为量阶。

计算时间复杂度的简化工具：两个定理

求和定理、求积定理

- **求和定理**：假定 $T_1(n)$ $T_2(n)$ 是程序 P1 P2 的运行时间，并且 $T_1(n)$ 是 $O(f(n))$ 的，而 $T_2(n)$ 是 $O(g(n))$ 的。那么，先运行 P1、再运行 P2 的总的运行时间是： $T_1(n) + T_2(n) = O(\text{MAX}(f(n), g(n)))$ 。

```
for (i=0; i<n; i++) a[i]=0;  //第一段
for (j=0; j<i; j++)  //第二段
    for (j=0; j<n; j++) a[i]= i+j;
```

按照求和定理为 $O(\max(n, n^2)) = O(n^2)$

计算时间复杂度的简化工具：两个定理

求和定理、求积定理

- **求积定理**：求积定理：如果 $T1(n)$ 和 $T2(n)$ 分别是 $O(f(n))$ 和 $O(g(n))$ 的，那么 $T1(n) \times T2(n)$ 是 $O(f(n) \times g(n))$ 的。

```
for (i=0; i<n; i++)  
    for (j=0; j<i; j++)  
        {s=s+i+j;    cout<<s;  }
```

按照求积定理，为 $O(n \times n) = O(n^2)$

算法的时间复杂度

当内外循环次数相互关联时，也可利用求积公式简化。

```
s=0;
for (i=0; i<n; i++)
    for (j=0; j<i; j++)
        {s=s+i+j;    cout<<s; }
```

按照求积定理，为 $O(n*n) = O(n^2)$

大 O 表示法的计算规则

- **规则 1:** 与问题规模无关的简单语句, 运行时间是一个常量, 时间复杂度为 $O(1)$ 。
 - **规则 2:** 条件语句, 除了条件判断, 分支以计算量大的为准。即 $\max(O(\text{then 子句}), O(\text{else 子句}))$ 。
 - **规则 3:** 循环语句, 因循环控制条件次数和循环体运行次数一致, 以循环体运行时间乘循环次数为准。
 - **规则 4:** 嵌套循环语句, 对外层循环的每个循环周期, 内层循环都要执行它的所有循环周期, 因此, 可用求积定理计算整个循环的时间复杂度。
- 连续语句: 利用求和定理把这些语句的时间复杂性相加。

算法的空间复杂度

算法的空间消耗包括三个方面：

- 实现算法的程序本身需要占据存储空间
- 待处理的数据需要在内存中存储，占据一定的空间
- 处理数据的过程中需要一些额外的辅助空间。

通常一和二是不可避免的，在设计算法时主要关注额外的辅助空间。

渐进空间复杂度也称空间复杂度：是当数据规模 n 趋于无穷时使用辅助空间的量阶，计为 $S(n)=O(f(n))$ 。

一个数据序列逆置的示例

Sequence Inversion

```
int a[10]={1,6,2,5,8,9,5,4,3,12};  
int t, i;  
for (i=0; i<5; i++)  
{  
    t=a[i];  
    a[i]=a[10-i-1];  
    a[10-i-1]=t;  
}
```

为完成 $n=10$ 个元素的逆置，
使用的辅助空间为一个变量 t ，
辅助空间数量和元素个数没有关系，
故其空间复杂度为 $O(1)$ 。

一个数据序列逆置的示例

Sequence Inversion

```
int a[10]={1,6,2,5,8,9,5,4,3,12},b[10];
int i;
for (i=0; i<10; i++)
    b[i] = a[10-i-1];
for (i=0; i<10; i++)
    a[i] = b[i];
```

使用了具有 n 个元素空间的数组 b 作为辅助空间，空间复杂度为 $O(n)$ 。

典型实例-最大连续子序列问题

- 给定（可能是负的）整数序列 $a_1, a_2, a_3, \dots, a_n$ ，寻找（并标识）其和最大的连续子序列。如果所有的整数都是负的，那么最大连续子序列的和是零。
- 例如，假设输入是 $\{-2, 11, -4, 13, -5, 2\}$ ，那么答案是 20，它表示连续子序列包含了第 2 项到第 4 项（如红色部分）；如果输入是 $\{1, -3, 4, -2, -1, 6\}$ ，答案是 7，这个子序列包含最后四项。

算法一：枚举法

分别求子序列

- $[0, 0], [0, 1], \dots, [0, n]$, 0 下标开始, 长度逐渐增大
- $[1, 1], [1, 2], \dots, [1, n]$, 1 下标开始, 长度逐渐增大
- ...
- $[n, n]$

的和, 求出最大值。

算法一：枚举法

算法一：MaxSubsequenceSum

```
int maxSubsequenceSum(int a[], int n, int &start, int &end)
{
    int i, j, k, thisSum, maxSum = 0;
    for (i = 0; i < n; i++)
        for(j = i; j < n; j++){
            thisSum = 0; //注意位置
            for( k = i; k <= j; k++)
                thisSum += a[ k ];
            if( thisSum > maxSum )
            {
                maxSum = thisSum; start = i; end = j; }
        }
    return maxSum;
}
```

算法一：枚举法

算法一：Main

```
int main()
{
    int a[]={-2, 11, -4, 13, -5, 2};
    int sum, start, end;
    sum = maxSubsequenceSum(a, 6, start, end);
    cout<< "From " << start << " to " << end << ": sum= " << sum <<endl;
    return 0;
}
```

算法一：时间复杂度分析

语句 `thisSum += a[k];`

- 在最里层循环中执行 $j - i + 1$ 次。
- 第二个循环的规模是 $n - i$ 。
- 最外层的循环规模是 n 。

因此最里层语句执行的次数是：

$$\sum_{i=0}^{n-1} \sum_{j=i}^{n-1} \sum_{k=i}^j 1 = (n^3 + 3n^2 + 2n) / 6$$

时间复杂度为 $O(n^3)$ 。

算法二： $O(n^2)$ 的算法

由于：

$$\sum_{k=i}^j a_k = a_j + \sum_{k=i}^{j-1} a_k$$

因此，方法一中最里层的循环可以省略。

内循环体中的语句 `thisSum += a[k]`；执行了 $n(n-1)/2$ 次

时间复杂度为： $O(n^2)$

算法二： $O(n^2)$ 的算法

算法二： MaxSubsequenceSum1

```
int maxSubsequenceSum1(int a[], int n, int &start, int &end)
{
    int i, j, k, thisSum, maxSum = 0;
    for (i = 0; i < n; i++)
    {
        thisSum = 0; //位置变了
        for(j = i; j < n; j++)
        {
            thisSum += a[ j ];
            if( thisSum > maxSum )
            {
                maxSum = thisSum; start = i; end = j; }
        }
    }
    return maxSum;
}
```

算法三: $O(n)$

- 如果一个子序列的和是负的, 则它不可能是最大连续子序列的首 (当然也不可能是尾) 部分, 因为我们可以选择不包含它来得到一个更大的连续子序列。
- 如: $\{-2, 11, -4, 13, -5, 2\}$ 的最大子序列不可能从 -2 开始, 因为子序列 $\{-2\}$ 的和是负的。
- $\{1, -3, 4, -2, -1, 6\}$ 的最大子序列不可能包含 $\{1, -3\}$ 。

算法三: $O(n)$

- 所有与最大连续子序列毗邻的连续子序列一定有负的（或 0）和（否则会包含它们）。
- 在算法二中，当检测出一个负的子序列时，不但可以从内层循环中跳出来，而且还可以让 i 直接增加到 $j+1$ ，因首部含这个负的子序列的所有序列都可以放弃。
- 如： $\{1, -3, 4, -2, -1, 6\}$ ，当检测序列 $[1, -3]$ 后，发现是负值，则表示该子序列不可能是最大子序列的首部。因此，接下去就可以从 4 开始检测。
- 事实上，如果一个子序列的和是零，且是一个子序列的首部，对子序列的和没有贡献，也可以舍弃。

算法三: $O(n)$

算法三: MaxSubsequenceSum2

```
int maxSubsequenceSum2(int a[], int n, int &start, int &end)
{
    int i, j = 0, thisSum = 0, maxSum = 0;
    for (i = 0; i < n; i++)
    {
        thisSum += a[i];
        if (thisSum <= 0) thisSum = 0, j = i + 1; //当前序列和不大于0, 舍弃, 从下一位开始
        if (thisSum > maxSum) {
            maxSum = thisSum;
            start = j;
            end = i;
        }
    }
    return maxSum;
}
```

- ① 什么是数据结构
- ② 算法及算法分析
- ③ C++ 部分概念
 - ① 面向过程与面向对象
 - ② 继承机制
 - ③ 泛型机制
 - ④ const 机制
 - ⑤ 异常处理

C++ 部分概念

- ① 面向过程与面向对象
- ② 继承机制
- ③ 泛型机制
- ④ const 机制
- ⑤ 异常处理

面向对象方法将**数据**和对**数据的基本操作处理函数**都封装在一个类中，分别成为一个类的**属性**和**成员函数**。

整个类相当于定义了一个新的**数据类型**，然后根据具体问题建立该类的对象（即变量），通过对象调用合适的函数来解决实际问题。

面向过程 VS 面向对象

任务：

将 1-20 之间的奇数存入数组，之后在这组数中查找用户输入的任意一个整数并报告查找结果。

find

```
void setValue(int b[], int n);  
void find(int b[], int n, int x); //查找x  
int main()  
{   int data[10], a;  
    setValue(data, 10);  
    cout<<"a=";    cin>>a;  
    find(data,10,a);  
    return 0;  
}
```

```
void setValue(int b[], int n)  
{   for (int i=0; i<n; i++)   b[i]=2*i+1; }  
  
void find(int b[], int n, int x)  
{   int i;  
    for (i=0; i<n; i++)   if (b[i]==x) break;  
  
    if (i==n)  
        cout<<x<<" doesn't exist in the  
            array!";  
    else  
        cout<<x<<" exists in the array!";  
    cout<<endl;  
}
```

函数与数据分离

面向对象

find

```
class arr
{ private:
    int *a;
    int maxSize, count;
public:
    arr(int size);
    void append(int x);
    void find(int x);
    ~arr(){delete []a;};    };
```

```
arr::arr(int size)
{
    a = new int[size];
    maxSize = size;
    count = 0;
}

void arr::append(int x)
{
    if (count==maxSize) return;
    a[count] = x;
    count++;
}
```

面向对象

find

```
void arr::find(int x)
{   int i;
    for (i=0; i<count; i++)
        if (a[i]==x) break;
    if (i==count)
        cout<<x<<" doesn't exist in the array!"
        ";
    else   cout<<x<<" exists in the array!";
    cout<<endl;
}
```

```
int main()
{   arr obj(10);
    int a;

    for (int i=0; i<10; i++)
        //将几个奇数放入对象
        obj.append(2*i+1);

    cout<<"a=";
    cin>>a;

    obj.find(a);
    return 0;
}
```

函数与数据包装在一起

C++ 部分概念

- ① 面向过程与面向对象
- ② 继承机制
- ③ 泛型机制
- ④ const 机制
- ⑤ 异常处理

数据结构的描述和实现

- 数据结构的**逻辑特性及基本操作**：每种数据结构用一个抽象类描述，指出该数据结构提供的**操作**，但不指明存储方式。——定义一个抽象类
- 数据结构的实现：每种数据结构可以有几种存储方式，即类中**属性描述**不同，因此要定义不同的类。但所有类都要符合数据结构的逻辑特性。为了保证这一点，最好每个类都从对应的抽象类继承。——定义一个子类（具体类）

SeqQue 的定义

```
class queue    //抽象类
{
public:
    virtual int find(int x)=0;
};
//父类是立规矩的！部分函数无法实现
//find是虚函数，子类必须实现

//顺序存储
class seqQue: public queue
{
private:
    int a[10];    //顺序存储
public:
    seqQue(int b[], int n);
    int find(int x);
};
```

```
//链式存储
struct Node
{
    int data;
    Node *next;
};
class linkQue: public queue
{
private:
    Node *head;    //链式存储
public:
    linkQue(int b[], int n);
    int find(int x);
};
```

SeqQue 的存储操作实现

//顺序存储

```
int seqQue::find(int x)
{
    for (int i = 0; i < 10; i++)
        if (a[i] == x) return 1;
    return 0;
}
```

//链式存储

```
int linkQue::find(int x)
{
    Node *p;
    p = head;
    while (p != NULL)
    {
        if (p->data == x) return 1;
        p = p->next;
    }
    return 0;
}
```

- 存储方式不同，基本操作的实现方法完全不一样

C++ 部分概念

- ① 面向过程与面向对象
- ② 继承机制
- ③ 泛型机制
- ④ const 机制
- ⑤ 异常处理

- 数据结构研究的是具有一定关系，且类型相同的一组元素。
- 元素类型不特指某种具体类型，如整型、字符型或者某种复杂的结构类型。
- 元素无论何种类型，它们在关系、基本操作处理方法上是一样的。

因此在数据类型上使用泛型机制：函数模板、类模板

- 数据结构更关注的是数据元素之间的关系是如何保存和处理的
- 所有的数据结构的类（包括抽象类和非抽象类）都是类模板
- 成员函数自动是函数模板

Template function

```
template <class T>
T Max(T a, T b)
{
    if (a > b) return a;
    return b;
}

int main()
{
    cout << Max(3, 5) << Max( 'a' , 's' ) << endl;
    return 0;
}
```

类模板的定义

```
template <class elemType>
class arr
{ private:
    elemType *a;
    int maxSize, count;
public:
    arr(int size);
```

```
//参数前加const, 保护参数在函数执行中不得修改
void append(const elemType &x);
```

```
//参数表后加const, 保护调用函数的对象值不得修
改
```

```
void find(const elemType &x) const;
~arr(){delete []a;};
};
```

类模板的定义

```
template <class elemType>
//类模板中成员函数自动为函数模板
arr<elemType>::arr(int size)
{
    a = new elemType[size];
    maxSize = size;
    count = 0;
}
```

```
template <class elemType>
void arr<elemType>::append(const elemType &x)
{
    if (count==maxSize) return;
    a[count] = x;
    count++;
}
```

类模板

类模板的定义

```
template <class elemType>
void arr<elemType>::append(const elemType &x)
{
    if (count==maxSize) return;
    a[count] = x;
    count++;
}
```

```
template <class elemType>
void arr<elemType>::find(const elemType &x)
    const
{
    int i;
    for (i=0; i<count; i++)
        if (a[i]==x) break;
```

```
        if (i==count)
            cout<<x<<" doesn't exist in the array!"
                ";
        else
            cout<<x<<" exists in the array!";
            cout<<endl;
    }
```

类模板

类模板的使用

```
int main()
{   arr<int> obj1(10);
    //<int>使得类模板实例化为一个模板类
    const arr<int> obj2(20);
    int a;
    const int b=100;
```

```
    for (int i=0; i<10; i++)
        obj1.append(2*i+1);
    cout<<"a=";   cin>>a;
    cout<<"In obj1: ";    obj1.find(a);
    cout<<"In obj1: ";    obj1.find(b);
    cout<<"In obj2: ";    obj2.find(a);
    return 0;
}
```

Template class

```
template <class T>
class queue    //抽象类
{
public:
    virtual int find(T x)=0;
};
```

Template class

顺序存储

```
template <class T>
class seqQue: public queue<T>
{
private:
    T a[10]; //顺序存储
public:
    seqQue(T b[], int n);
    int find(T x);
};
```

成员函数成为函数模板

```
template <class T>
int seqQue<T>::find(T x)
{
    for (int i = 0; i < 10; i++)
        if (a[i] == x) return 1;
    return 0;
}

template <class T>
seqQue<T>::seqQue(T b[], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = b[i];
}
```

Template class

链式存储

```
template <class T>
struct Node
{
    T data;
    Node *next;
};
template <class T>
class linkQue: public queue<T>
{
private:
    Node<T> *head; //链式存储
public:
    linkQue(T b[], int n);
    int find(T x);
};
```

类模板

链式存储的两个成员函数的实现

Template class

```
template <class T>
linkQue<T>::linkQue(T b[], int n)
{
    Node<T> *last = new Node<T>;
    this->head = last;
    last->next = NULL;
    for (int i = 0; i < n; i++)
    {
        last->data = b[i];
        Node<T> *tmp = new Node<T>;
        tmp->next = NULL;
        last->next = tmp;
        last = tmp;
    }
}
```

```
template <class T>
int linkQue<T>::find(T x)
{
    Node<T> *p;
    p = head;
    while (p != NULL)
    {
        if (p->data == x) return 1;
        p = p->next;
    }
    return 0;
}
```

C++ 部分概念

- ① 面向过程与面向对象
- ② 继承机制
- ③ 泛型机制
- ④ `const` 机制
- ⑤ 异常处理

- 用 `const` 修饰变量，变量将变为常量。
- 常量一般有初值，在常量的生命周期中，终身不得改变其值。

C++ 中 `const` 最普通的用法：`const double PI = 3.14;`

类定义时常见的两种 const 用法

1. 函数 `void find(const elemType &x)const` 中修饰参数 `x` 的 `const` 和 `&` 组合。
因为参数类型 `elemType` 是一种泛型类型，调用 `find` 函数时实参可能是复杂的结构类型，加了 `&` 符号就有了明显的好处：
 - a) 形参 `x` 不分空间，节省了空间消耗。
 - b) 没有空间分配，在实参和形参打交道时，就不涉及拷贝构造函数的执行。提高了运行效率。
- 故泛型类型参数前尽量加 `&` 符号，如果函数又不需要改变这个参数，请养成在参数前加 `const` 和 `&` 组合的习惯。

类定义时常见的两种 const 用法

1. 函数 `void find(const elemType &x)const` 中参数表后 `const` 的用法。

参数表后的 `const` 是保护调用它的对象的值不能被改变，即对隐含参数 `this`，保护 `*this` 中的内容，使其不能被修改。

语句 `obj1.find(a)`，这个 `const` 就是保护对象 `obj1` 的值。

一个成员函数不准备改变调用它的对象的值，请养成在参数表后加 `const` 的习惯。

如果没有函数参数表后带 `const` 的支持，常对象 `obj2` 是不能调用这个函数的。

常对象 `obj2` 只能调用参数表后带 `const` 的成员函数，加 `const` 就为常对象调用它开一扇窗。

```
const arr obj2; obj2.find(10);
```

类模板

类模板的使用

```
int main()
{   arr<int> obj1(10);
    //<int>使得类模板实例化为一个模板类
    const arr<int> obj2(20);
    int a;
    const int b=100;
```

```
    for (int i=0; i<10; i++)
        obj1.append(2*i+1);
    cout<<"a=";   cin>>a;
    cout<<"In obj1: ";    obj1.find(a);
    cout<<"In obj1: ";    obj1.find(b);
    cout<<"In obj2: ";    obj2.find(a);
    return 0;
}
```

量：变量、常量

加 const 保护数据：

- ① 成员函数参数表后加 const：函数不会修改对象的状态
- ② 参数前 const & 组合：该参数不会在函数中被修改

实例

Const 用法实例 1

```
template <class elemType>
class arr
{
private:
    elemType *a;
    int maxSize, count;
public:
    arr(int size);
    void append(const elemType &x);
    //参数表后加const, 保护
    //调用函数的对象的值不得修改
    void find(const elemType &x) const;
    ~arr(){delete [] a;};
};
```

```
int main()
{
    arr<int> obj1(10);
    const arr<int> obj2(10);
    // 将1, 3, 5, ... 等奇数放入对象
    for (int i = 0; i < 10; i++)
        obj1.append(2*i+1);
    int a;
    cin >> a;
    cout << "a=" << a << endl;
    obj1.find(a);
    obj2.find(a);
    return 0;
    //obj2为const的类, 调用的find需要加上
    const修饰。
}
```

实例

Const 用法实例 2

```
template <class elemType>
class arr
{
private:
    elemType *a;
    int maxSize, count;
public:
    arr(int size);
    void append(const elemType &x);
    //参数表后加const, 保护
    //调用函数的对象的值不得修改
    void find(const elemType &x) const;
    ~arr(){delete [] a;};
};
```

```
int main()
{
    arr<int> obj1(10);
    const int b = 10;
    // 将1, 3, 5, ... 等奇数放入对象
    for (int i = 0; i < 10; i++)
        obj1.append(2*i+1);
    int a;
    cin >> a;
    cout << "a=" << a << endl;
    obj1.find(a);
    obj1.find(b);
    return 0;
    //obj1的函数find因为定义了const变量x所以
    //可以搜索b, 否则不可以。为了保证b不在
    //调用函数中被修改。
}
```

C++ 部分概念

- ① 面向过程与面向对象
- ② 继承机制
- ③ 泛型机制
- ④ const 机制
- ⑤ 异常处理

异常处理

在数组类模板中增加一个函数 `fetch`，它读取并返回下标为 `i` 的元素

考虑问题：

当 `i` 越界时返回什么合适？

调用 `exit` 函数？整个应用程序结束，小题大做！没给用户处理机会。

fetch

```
template <class elemType>
elemType arr<elemType>::fetch(int i) const
{
    if ((i<0) || (i>=count))
    {
        cout<<"index is out of the bound!"<<
            endl;
        exit(1);
    }
    else
        return a[i];
}
```

异常处理方式-给用户处理的机会

throw, try, catch 异常处理机制

class tooSmall{};

class tooBig{};

? 垃圾分类, 两类垃圾

fetch

```
template <class elemType>
elemType arr<elemType>::fetch(int i) const
{
    if (i<0)
        throw tooSmall();
    if (i>=count)
        throw tooBig();
    else
        return a[i];
}
```

实例

异常处理

```
int main()
{   arr<int> obj1(10);
    const arr<int> obj2(20);
    int i;

    try {
        for (i=0; i<10; i++)
            obj1.append(2*i+1);
```

```
while (cin>>i)
    cout<<i<<": "<<obj1.fetch(i)<<
        endl;
}
catch (tooSmall){cout<<"index is too small
!"<<endl;}
catch (tooBig){cout<<"index is too big!"<<
    endl;}

cout<<"Return to main, it is Over! "<<endl
    ;
return 0;
}
```

思考：如果只用了 throw, 没有用 try catch, 会怎样？在哪一层上用 try catch？

Arr 类的定义

```
template <class elemType>
class arr
{
private:
    elemType *a;
    int maxSize, count;
public:
    arr(int size);
    void append(const elemType &x);
    void find(const elemType &x) const;
    elemType &get(int i) //下标为i的元素或单元
    ~arr(){delete [] a;};
};
```

异常处理

Arr 类的定义与成员函数 get 的实现

```
//用户定义错误类型  
class errType1{};  
class errType2{};
```

在 arr 类中增加一个成员函数 get

```
template <class elemType>  
elemType &get(int i)  
{  
    if (i < 0) throw errType1();  
    if (i > count) throw errType2();  
    return a[i];  
}
```

解决了下标越界时，没有合适返回值的尴尬

异常处理：例子

异常处理实例

```
#include <bits/stdc++.h>
using namespace std;
class errType1{};
class errType2{};
template <class T>
T get(T a[], int n, int i)
{
    if(i < 0) throw errType1();
    if (i >= n) throw errType2();
    return a[i];
}
```

```
int main()
{
    int s[10]={8, 7, 5, 9, 6, 3, 2, 6, 14, 25};
    try
    {
        cout << get(s, 10, 1) << endl;
        cout << get(s, 10, -5) << endl;
        cout << get(s, 10, 2) << endl;
        cout << get(s, 10, 20) << endl;
        cout << get(s, 10, 3) << endl;
    }
    catch (errType1)
    {cout << "less than 0" << endl;}
    catch (errType2)
    {cout << "too big" << endl;}
    cout << "work in main!" << endl;
    return 0;
}
```

总结

- 数据元素及元素间关系称作数据结构。
- 数据结构研究具有某种制约关系的一组元素及元素间关系在内存中如何存储、在各种存储方式下基本操作如何实现，以及各种数据结构的典型应用。
- 具体研究分逻辑结构及基本操作、物理结构、基本操作实现、典型应用四个方面。
- 在分析逻辑结构和基本操作时，要完全脱离计算机而仅仅依赖现实生活中的元素特征来分析元素、元素关系及基本操作，最后给出用伪代码描述的抽象数据类型。
- 在物理结构分析阶段，讨论元素及元素关系在内存中如何存储。存储可以分顺序存储和链式存储，顺序存储使用一块连续的空间存储元素和元素之间的关系；链式存储使用各自独立的空间存储每个元素，并在每个独立的空间中附加字段以存储元素之间关系。
- 在基本操作实现分析阶段，研究在各种存储方式下基本操作的实现方法和步骤即算法。对于算法提出了时间复杂度和空间复杂度的概念及计算方法，并以此为依据，对不同算法进行性能比较。

- 数据结构的基本概念
- 算法的确定比程序设计技巧对程序运行时间的影响更大
- 大 O 分析是一种很有效的工具，但它也有局限性。如前所述，它的使用不适合输入规模较小的场合。此时，最高项的系数或低次项对运行时间的影响较大。

编程常见问题

选择代码开发平台 (IDE)

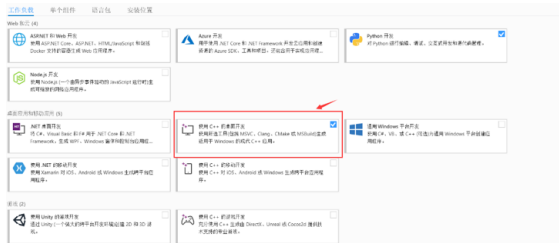
• CodeBlocks 20.03

- CodeBlocks 相关问题汇总
 - Q & A
 - 1. 如何解决中文乱码问题
 - 2. 如何修改编译器类型
 - 3. 如何设置编译器路径
 - 4. 如何开启 C++11 或者更新的 C++ 标准
 - 5. 首页跳出的 Tip of the Day 是什么
 - 6. 如何设置 Debug / 调试器 路径
 - 7. 侧边栏消失了怎么办?
 - 8. 编译输出栏消失了怎么办? 或者我的xxx布局消失了找不到了怎么办?
 - 9. CodeBlocks 如何添加多文件到项目中?
 - 下载 `codeblocks-20.03mingw-setup.exe`
 - 安装
 - 第一次使用
 - 文件关联 CodeBlocks 的提示
 - 新建一个C++项目
 - CodeBlocks 如何进行简单的代码调试 / Debug ?
 - 汉化

<https://www.codeblocks.org/downloads/>

选择代码开发平台 (IDE)

- Visual Studio 2019 社区版



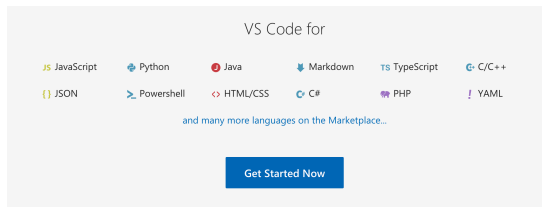
<https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&rel=16>

- 提示:

如果选择安装 VS2019, 注意需要勾选 “Desktop development with C++” / “使用 C++ 的桌面开发”

选择代码开发平台 (IDE)

- Visual Studio Code (VS Code)



<https://code.visualstudio.com/>

- 相较于其他 IDE, VS Code 的优势:
 - 轻量级, 安装包小, 且启动速度快。
 - 拥有丰富的插件系统。
 - 具有代码跟踪功能, 可以对代码进行标注。

常见 C++ 代码问题

常见问题:

<https://gitee.com/OneForward/TACpp/blob/gitee/tutorials/Cpp.md>

- 变量或者对象的值没有**初始化**，或者类内某些成员没有初始化。
- 数组没有初始化却被访问。（可能引发 SIGSEGV 内存异常的中断）
- 数组**越界**访问。（可能引发 SIGSEGV 中断）
- **指针没有 new 就访问**。此时指针值是随机值或者 NULL，访问该位置的内存会引发 SIGSEGV 中断。
- **指针 delete 后再被访问或者再试图 delete**。此时指针值所在的内存位置已经没有对象，任何访问都将引发 SIGSEGV 中断。
- 对象在生存期内，可能被多个指针共享。假若某个指针在某个地方删除了对象，然后别的指针操作该对象时就可能出错。

常见 C++ 代码问题

常见问题：

<https://gitee.com/OneForward/TACpp/blob/gitee/tutorials/Cpp.md>

1) 编程遇到 bug，常思考：

- 我的变量都初始化了吗？（未初始化对象可能为随机值）
- 我的数组初始化了吗？
- 访问数组时，访问了没有初始化的位置了吗？
- 数组访问越界了吗？

常见 C++ 代码问题

2) 初始化数组全部为默认值的方法

实例：初始化数组全为默认值

```
const int N = 1024;  
int arrA[N] = {}; //默认全部为0  
bool arrB[N] = {}; //默认全部为 false  
char arrC[N] = {}; //默认全部为 '\0'  
//或者写一个 for 循环赋值，或者使用 memset  
//C++11之后上述等号也可以全部删掉，甚至还有  
int *pArr = new int[N] {};
```

指针与常见问题

1) 指针未指向合法内存位置就访问

实例：指针未指向合法内存位置

//错误写法1

```
int* p; // p points to random position of memory
*p = 1; // trouble
```

//错误写法2

```
int* p;
p = &1; //compiler error, 取地址运算符只能作用于左值（例如变量）
```

指针与常见问题

2) 指针回收相关

实例：错误指针回收

//错误写法1

```
int* p; //p points to random position of memory
```

```
delete p; //trouble
```

//错误写法2

```
void f() {
```

```
    int* p = new int[100];
```

```
    delete [] p;
```

```
    //... wait a while ...
```

```
    delete [] p; //error, 不能回收一个已经被回收的内存
```

```
}
```

//错误写法3

```
int* p1 = new int { 99 };
```

```
int* p2 = p1; //potential trouble
```

```
delete p1; //now p2 doesn't point to a valid object
```

```
p1 = nullptr; // gives a false sense of safety
```

```
char* p3 = new char { 'x' };
```

```
*p2 = 999; //error, 试图访问已被回收的内存
```

```
cout << *p3;
```

//错误写法4

```
//指针 p 指向某个对象
```

```
delete p; // 回收该对象
```

```
if (p == nullptr) { //错误, 指针p的值未必为空  
    //... something ...
```

```
}
```

指针与常见问题

3) 指针的声明问题

实例：指针声明

```
Node* p1, p2, p3; //注意 p2, p3 类型不是 Node*
```

4) 指针赋值是地址值的传递。体会差别：

实例：指针赋值

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
Node* head = new Node;  
//method1  
Node* next = head->next; next = new Node;  
与  
//method2  
head->next = new Node;
```

只有方法 2 可以正确的构造新节点

函数的参数与类型签名

1) 数组如何做函数参数

数组做参数时会直接被隐式转化为指针。如下声明是等价的

Case

```
void f(int []);  
void f(int *);
```

函数的参数与类型签名

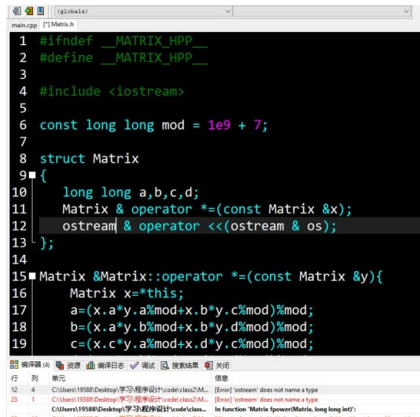
2) 前置函数声明，在后面实现函数时，函数签名不匹配。('const', '&' 修饰符)

Case

```
//例如前置的函数声明
bool search(const int&) const;
//后置函数实现
//void search(const int&) const {...}; 错误，函数签名不匹配
//bool search(int&) const {...}; 错误，函数签名不匹配
//bool search(const int) const {...}; 错误，函数签名不匹配
//bool search(const int&) {...}; 错误，函数签名不匹配
bool search(const int&) const {...}; //OK
```

多文件经典问题

1) 引入头文件忽略了使用 'std' 命名空间, 例如



```
1 #ifndef __MATRIX_HPP__
2 #define __MATRIX_HPP__
3
4 #include <iostream>
5
6 const long long mod = 1e9 + 7;
7
8 struct Matrix
9 {
10     long long a,b,c,d;
11     Matrix & operator *=(const Matrix &x);
12     ostream & operator <<(ostream & os);
13 };
14
15 Matrix &Matrix::operator *=(const Matrix &y){
16     Matrix x=*this;
17     a=(x.a*y.a%mod+x.b*y.c%mod)%mod;
18     b=(x.a*y.b%mod+x.b*y.d%mod)%mod;
19     c=(x.c*y.a%mod+x.d*y.c%mod)%mod;
```

行	列	单元	信息
12	4	C:\Users\19588\Desktop\学习\程序设计\code\class2\M...	[Error] 'ostream' does not name a type
23	1	C:\Users\19588\Desktop\学习\程序设计\code\class2\M...	[Error] 'ostream' does not name a type
		C:\Users\19588\Desktop\学习\程序设计\code\class...	In function 'Matrix (power(Matrix, long long int))':

这里要么使用 'std::ostream', 要么引入 'std' 命名空间

多文件经典问题

2) 头文件里面声明了某些函数，但是没有在相应的.cpp 文件里实现这些函数，会提示 ‘undefined reference’

列	单元	信息
	C:\Users\19588\Desktop\学习\程序设计\code\class\...	In function 'main':
	C:\Users\19588\Desktop\学习\程序设计\code\class\ma...	undefined reference to 'Thermometer::~Thermometer()'
	C:\Users\19588\Desktop\学习\程序设计\code\class\ma...	undefined reference to 'Thermometer::~Thermometer()'
	C:\Users\19588\Desktop\学习\程序设计\code\class\col...	[Error] ld returned 1 exit status
	C:\Users\19588\Desktop\学习\程序设计\code\class\Ma...	recipe for target '项目1.exe' failed

例如这里的错误的原因是头文件里面声明了析构函数，但作者忘记了实现它（头文件里面不写明的话编译器则会帮你实现

> 同时要注意函数声明与定义时的签名是否一致。见 [函数的参数、返回值与类型签名](#函数的参数返回值与类型签名)

C++ 的预编译宏

- #include 宏

#include <filename>

#include "filename"

#include 宏的本质是包含一个文件并在当前位置插入。本质仍是一个文本替换。一般自己写的头文件使用**引号**引入。